MAZE SOLVER

USING A* ALGORITHM

REPORT

UTSAV MAKHIJA
25BCE10713

INTRODUCTION

Artificial Intelligence plays a crucial role in solving complex decision-making problems efficiently. One such problem involves pathfinding, where an optimal route must be determined between two given points while avoiding obstacles. This project focuses on building an AI-based Maze Solver that demonstrates how intelligent algorithms can be used to solve such problems. The system uses the A* (A-Star) Search Algorithm to find the shortest path in a randomly generated maze.

PROBLEM STATEMENT

How can an intelligent system efficiently find the shortest path from a start point to an endpoint in a maze containing obstacles?

The system must:

- Explore multiple possible paths
- Avoid obstacles
- Choose the most optimal (shortest) route

Efficient pathfinding ensures:

- Reduced time and cost
- Better resource utilization

APPROACH

To solve this problem, I used A* Search Algorithm, which is widely used for pathfinding. the algorithm:

- Uses a heuristic function to estimate the best path
- Prioritizes nodes that are more likely to lead to the goal
- Balances between:
    - Actual cost from start (g(n))
    - Estimated cost to goal (h(n))

Formula Used : $f(n) = g(n) + h(n)$

Where:

- $g(n)$ = distance from start node
- $h(n)$ = heuristic cost

DESIGN

I. Maze Representation:

The maze is represented as a 2D grid Each cell can be:

- Empty (walkable)
- Wall (blocked)

II. Key Components:

- Start Node → Top-left corner
- End Node → Bottom-right corner
- Neighbors → adjacent cells (up, down, left, right)

III. Visualization:

The system visually represents:

- Explored nodes
- Current path
- Final shortest path

FLOW OF ARCHITECTURE

User Input (UI)
↓
Maze Generator
↓
A* Algorithm Engine
↓
Visualization Module
↓
Output (Path + Stats)

WORKING

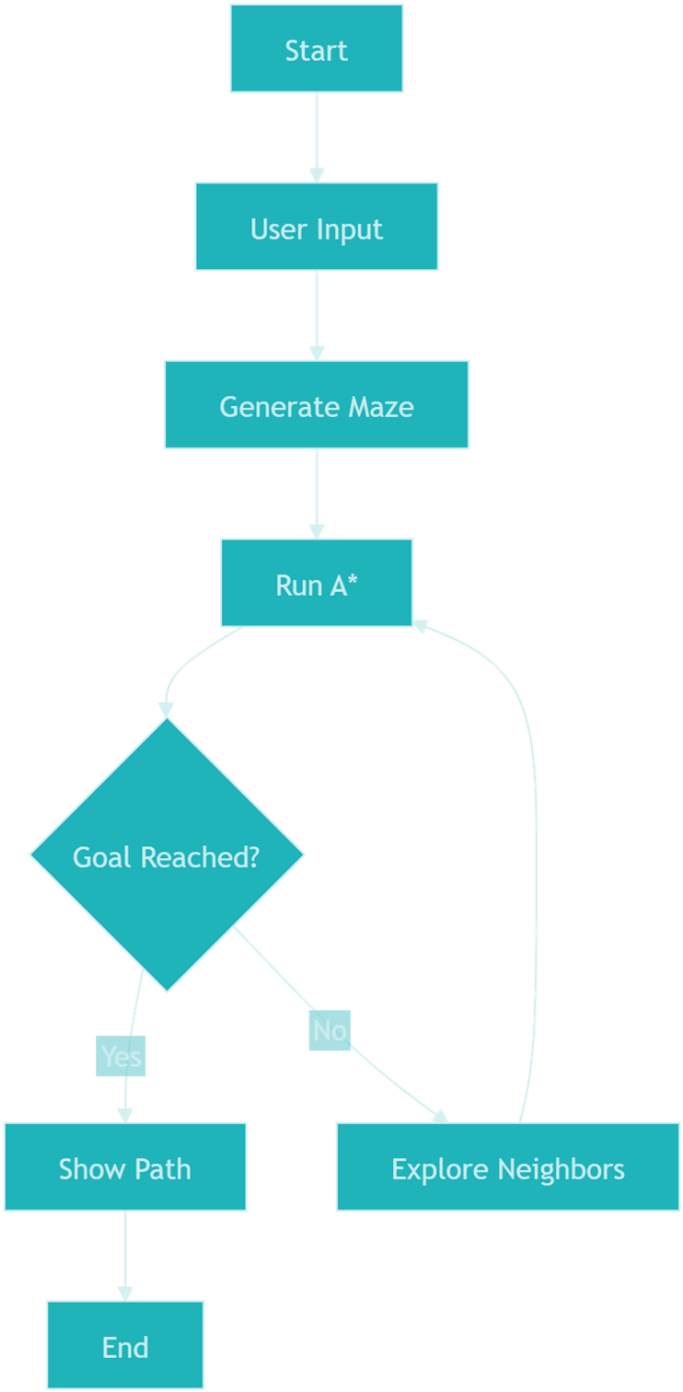
The working of the project can be broken down into the following steps:

I. Maze Generation:
    a. A random grid is generated with obstacles
II. Initialization:
    a. Start and end points are defined
    b. Data structures are initialized
III. Pathfinding Begins:
    a. A* algorithm starts exploring nodes
IV. Node Evaluation:
    a. For each node:
        i. Calculate cost
        ii. Compare with alternatives
        iii. Choose optimal node
V. Visualization:
    a. Explored nodes are highlighted in real time
VI. Path Reconstruction:
    a. Once the goal is reached, backtrack to find the shortest path
VII. Display final path.

TOOLS AND TECHNOLOGIES

- Python → Core programming language
- Streamlit → Interactive UI and visualization
- NumPy → Efficient grid handling
- Heapq → Priority queue for A* implementation

WORKFLOW DIAGRAM



DESCISIONS MADE

1. Choosing A* algorithm:
    - Faster and more efficient than BFS and DFS
    - Guarantees shortest path
2. Grid-based Maze:
    - Easy to implement and visualize
    - Matches real-world pathfinding scenarios
3. Real-time Visualization:
    - Helps in understanding algorithm behavior

CHALLENGES FACED

1. Visualization Issues
    - Initially, the maze appeared as plain text ("empty")
    - Solved by converting grid into a color image
2. Performance Problems
    - Large grid sizes caused slow execution
    - Resolved by : Reducing grid size
3. Image Quality
    - Output appeared pixelated
    - Fixed by scaling each grid cell to higher resolution
4. Debugging Errors
    - Faced issues with data types (e.g., numpy vs list)
    - Learned importance of consistent data structures

KEY LEARNINGS

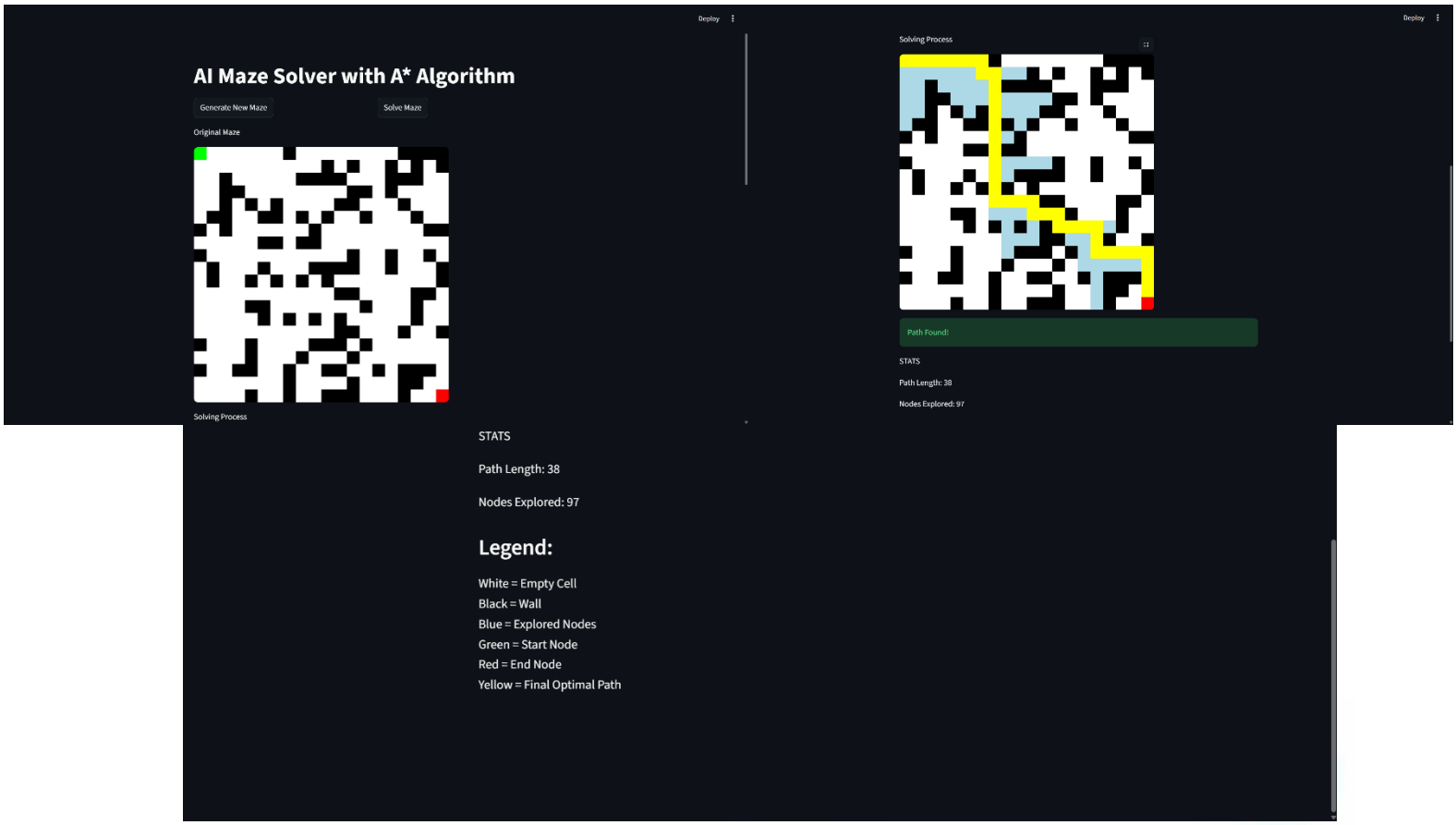
This project helped me understand:

- How AI search algorithms work in practice
- Importance of heuristics in optimization
- How to visualize algorithms for better understanding
- Building interactive applications using Python

It also improved my ability to:

- Break down complex problems
- Make design decisions
- Optimize code

OUTPUTS



REAL WORLD APPLICATIONS

The same concept can be applied in:

- GPS Navigation Systems
- Autonomous Robots
- Video Game AI
- Delivery Route Optimization
- Warehouse Automation
- Emergency Route Planning

FUTURE ENHANCEMENTS

The project can be further improved by:

- Adding multiple algorithms (BFS, DFS)
- Allowing user-drawn mazes
- Supporting diagonal movement
- Adding performance comparison graphs and metrics
- Converting into a full interactive game
- Deploying as a web application

CONCLUSION

The AI Maze Solver successfully demonstrates how intelligent algorithms can solve pathfinding problems efficiently. By using the A* algorithm, the system is able to find the shortest path while minimizing unnecessary exploration.

This project bridges the gap between theoretical AI concepts and practical implementation, making it a strong foundation for further exploration in Artificial Intelligence and Machine Learning.